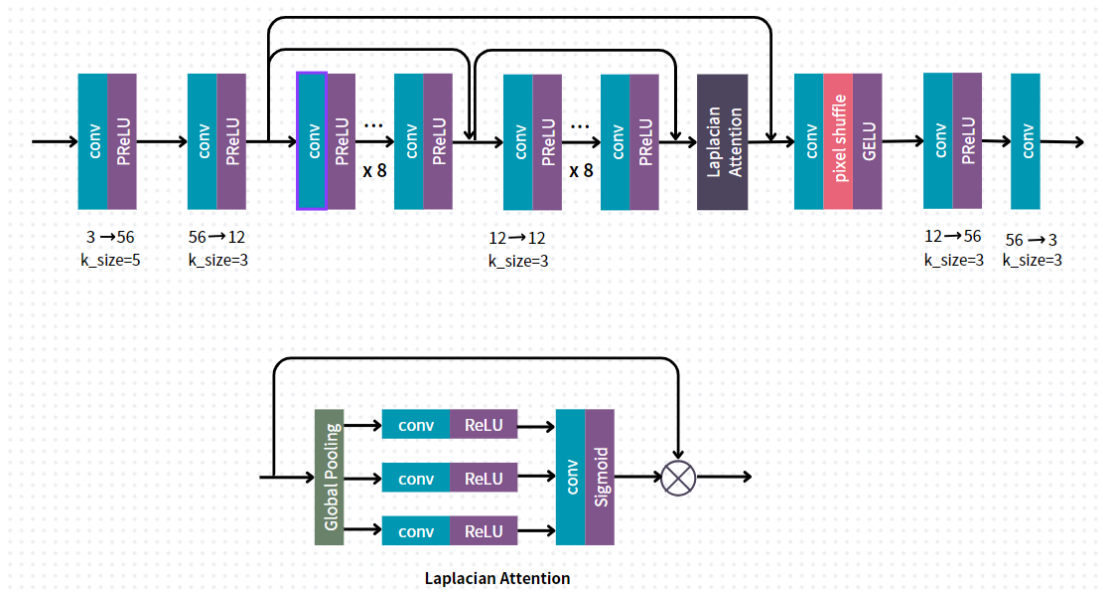


# Final Project

## Model Architecture and Training

### 1. Model Architecture



### 2. Training Techniques

#### 1) Dataset:

- (1) epoch 1-5: DIV2K \* 6
  - (2) epoch 6-10: DIV2K \* 6 + Flickr2K
  - (3) epoch 11-15: DIV2K \* 6 + Flickr2K(rotate 90°)
- DIV2K \* 6: rotate 90° 、vertical flip 、horizontal flip

#### 2) input image size: 16 x 16

#### 3) batch size: 16

#### 4) epochs: 15

### 3. Results

```
Number of LR imgs (patches): 5 , Number of HR imgs (patches): 5
=====EVALUATION=====
Testing: baby.png , PSNR: 32.6803092956543
Testing: bird.png , PSNR: 30.329484939575195
Testing: butterfly.png , PSNR: 23.640857696533203
Testing: head.png , PSNR: 30.08957862854004
Testing: woman.png , PSNR: 28.062076568603516

Average PSNR: 28.960 dB

/usr/local/lib/python3.10/dist-packages/thop/vision/calc_func.py:53: User
warnings.warn("This API is being deprecated")
=====
FLOPs : 7.563394264 G
PARAMS : 0.048023 M
=====
Your FLOPs is larger than 11.1 G.
You will not get this 10 points.
=====
Overall Ranking Score: 21444.328
=====
Your score on Kaggle should be 541.832581
=====
```

#### 4. idea:

- (1) 參考 CoAtNet 的想法，先做 convolution，後面接 attention，可以互補兩這的優缺點。
- (2) 最一開始試想使用 hat 的 model，因為在使用 DIV2K dataset 跑 10 個 epoch 時，這一個 model 的表現最好。但是考慮到計算量太大，最後決定使用計算量最少的 FSRCNN。
- (3) 使用 Residual in Residual，將 convolution 8 個 8 個連接，最後再將中間層的前後連接起來。

### Process

#### 1. Training Method

- 1) 方法一:使用 RandomCrop 將每一張圖片切成 16 x 16 進行訓練。
  - 2) 方法二:
    - (1) 用 flip pad 把 low resolution 的圖片 pad 成 680 x 680，high resolution 的圖片 pad 成 2040 x 2040
    - (2) 把圖片切小放進 model 放大後再把放大過的圖片組起來計算 loss問題點:
    - (1) 需要另外的 memory 空間來儲存切小後的 low resolution 圖片和放大過後的 high resolution 圖片。
    - (2) 很容易造成 cuda out of memory，訓練不完
  - 3) 方法三:
    - (1) 用 flip pad 把 low resolution 的圖片 pad 成 680 x 680，high resolution 的圖片 pad 成 2040 x 2040
    - (2) 把圖片切小放進 model，每一個切小後的 tile 都拿來算 loss問題點: 同樣會造成 cuda out of memory
- 原本想用方法二和三，可以增加 dataset 數量，也比較可以有效運用 data，但是因為可能因為會用到雙層 for 迴圈，加上 data 數會增加很多，因此在有限的 memory 下無法使用。

#### 2. Input Image Size

- 最後決定使用 16 x 16 作為 input image size，避免 cuda out of memory。
- 1) input image size 越大，loss 下降的越快。
  - 2) input image size 越大，越容易造成 cuda out of memory。
  - 3) 嘗試的解決 cuda out of memory:
    - (1) 方法: 在圖片進入 model 後先把圖片切成 64 x 64 再用 for 迴圈跑各個 layer結果: 計算量幾乎沒變。可能是因為有用到 for 迴圈，所以計算量也沒有變。

(2) 方法: 在進入 model 之後先 resize

(3) 結果: 計算量反而增加。可能是 resize 的計算量真的很大。且 super resolution 的目的是要放大圖片且增加解析度，圖片先縮小再放大會導致解析度變差，也不適合這次的 task。

### 3. Dataset

最終將 epoch 分為三段，分別使用不同的 dataset。參考了 FSRCNN 論文中的訓練方法，先使用少量的 data，在 loss 開始停止下降時增加 data 數量。

- 1) epoch 1-5: DIV2K \* 6: 用 data augmentation 擴充 data 數量，為了省時，先使用  $800 * 6 = 4800$  個 data。
- 2) epoch 6-10: DIV2K \* 6 + Flickr2K: 在 loss 下降速度開始停滯時增加 data 到  $4800 + 2650 = 7450$  個 data。
- 3) epoch 11-15: DIV2K \* 6 + Flickr2K(rotate 90°): 因為我事先將跑完 10 個 epoch 後的 weight 存起來，之後再 load 這個 weight 多訓練 5 個 epoch，為了避免這 5 個 epoch 的 data 和第 6-10 個 epoch 的 data 一樣會重複訓練，因此決定加上 rotate 90° 的 data augmentation。

### 4. Data Augmentation

太相近的 data 對 model 會是反效果。為避免和 vertical flip 出來的結果太相近，rotate 只使用 90°，並將 rotate、horizontal flip、vertical flip 互相搭配組成六種 dataset。

### 5. Get Low Resolution Images

- (1) DIV2K: 直接使用 dataset 中的 low resolution images
- (2) Flickr2K: 使用 transforms.RandomCrop.get\_params 取得 low resolution random crop 的參數後，乘三 crop high resolution 的圖片。

### 6. learning rate:

- (1) 固定為 1e-3，last part(最後兩個 convolution)固定為 1e-4。
- (2) 最一開始我是使用 exponential learning rate，起始 learning rate 為 1e-3，gamma 為 0.7 但 PSNR 一直停止在 6 到 10 之間。之後改為 1e-3 固定不下降後，PSNR 就上升到 27。可能是一開始的條件 learning rate 降得太快，後面 loss 下降速度就會很慢，導致 PSNR 升不起來。
- (3) 之後雖然有試過中途將 learning rate 減小，但是固定在 1e-3 時，loss 下降的最快。
- (4) 若 learning rate 調到 1e-2，初始的 loss 會非常高。

### 7. Base Models

learning rate: 1e-3

- 1) Model 1: 參考 FSRCNN，將 deconstruction 的部分改為 pixel shuffle，並在

mid\_part 加上 residual connection 和 Laplacian Attention。

2) Model 2: HAT

3) Model 3: SwinIR

4) Model 3: DRLN 參考 FSRCNN 將中間的 channel 數調小

| Model | FLOPs(G) | PARAMS(M) | PSNR   |
|-------|----------|-----------|--------|
| 1     | 7.5634   | 0.048     | 28.655 |
| 2     | 695.473  | 5.285     | 29.555 |
| 3     | 461.485  | 3.492     | 28.822 |
| 4     | 28.2513  | 0.222     | 26.599 |

learning rate: ExponentialLR(1e-3, gamma=0.7)

1) Model 1:

(1) 參考 FSRCNN，將 deconstruction 的部分改為 pixel shuffle，並在 mid\_part 加上 residual connection 和 Laplacian Attention。

2) Model 2:

(1) 比 Model 1 少了 Laplacian Attention

(2) 比 model 1 層數少，residual connection 也較少，但是計算量卻差不多。

3) Model 3:

(1) 架構: 參考 CoAtNet 的架構，將 hat 的 embed dimension 和深度等皆減到約一半，並在最前面加兩層 Residual convolution。

4) Model 4:

(1) 架構: 將 model 1 的 convolution 層刪除，並將各個 block 加深一層。

(2) 由表三可看出，將前面的 convolution 層刪除，即便在參數量和計算量較多的情況下，PSNR 仍較低。

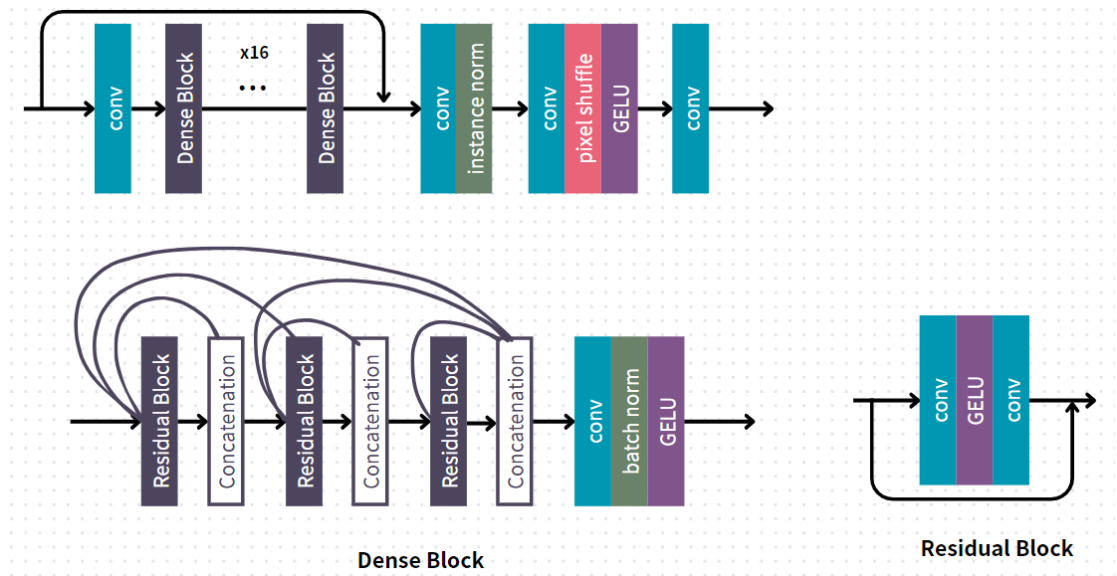
(3) 由表三可看出使用 hat model，能夠在比 CNN base model 少計算量的情況下跑出較好的結果。但是在調小 model 的時候，個個參數都調到非常小，計算量也比 60 還高。例如在 image size=32, embed\_dim=36, depth=(1, 1), heads=(3, 3), window size=4 的情況下，計算量仍為 63。

5) Model 5:

(1) 架構: 參考 DLSN 的 dense layer，每一個 dense layer 皆由兩個 Residual block 組成，如下圖。

(2) 想法: concatenation 對 PSNR 的提升很有用，如果可以再把這個模型再加深一點，可能會有較好的結果。

(3) 結果: 計算量和參數量都比其他 model 大非常多，但是 PSNR 比其他 model 小。



#### 6) Model 6:

- (1) 架構: 使用 DSLN 的 dense blocks，不加 Laplacian Attention。
- (2) 本來想說 super resolution 較像是局部的 task，或許看全局的 attention 可能沒有那麼重要。但是和 model 5 和 model 6 相比可知 attention 不但可以提升 PSNR，也能減少計算量和參數量。

#### 7) Model 7:

- (1) 架構: 參考 RCAN，並在過程中像 FSRCNN，減小 channel 數。
- (2) 因為中間有減小 channel 數，所以 PSNR 較低。後來有嘗試加深 model，但是沒有加深很多 FLOPs 就增加到和其他 model 差不多了。

#### 8) Model 8:

- (1) 架構: 參考 DRLN，layer 數由 20 減小為 9，dense connection 的層數由改為 2。

#### 9) Model 9:

- (1) 架構: 和 model 4 相似，但把 Laplacian Attention 改為 RCAN 的 Channel Attention。
- (2) 由表三可看出依照原本的 DSLN 使用 Laplacian Attention，效果較好。

#### 10) Model 10:

- (1) 架構: 參考 RCAN，但總層數由 10 減小為 6，個別 Residual Block 中模塊的個數由 20 減小為 3，且將 Channel Attention 改為 DRLN 的 Laplacian Attention。
- (2) 這一個 model 在 train 的時候中途斷線，但因為他的 validation loss 後面不斷上升，因此不考慮此 model。由此可見 dense layer 的計算量和參數量都非常大，但也相當有效果。

| Models | FLOPs(G) | PARAMS(M) | PSNR   |
|--------|----------|-----------|--------|
| 1      | 36.508   | 0.275     | 6.433  |
| 2      | 36.507   | 0.267     | 6.438  |
| 3      | 202.444  | 1.526     | 10.689 |
| 4      | 219.220  | 1.653     | 10.502 |
| 5      | 851.477  | 6.441     | 6.560  |
| 6      | 850.328  | 6.132     | 7.129  |
| 7      | 128.564  | 1.003     | 6.433  |
| 8      | 630.480  | 4.926     | 8.259  |
| 9      | 630.479  | 4.806     | 7.821  |
| 10     | 253.672  | 2.174     | 中途斷線   |

表三

以上結果的 training 條件皆為:

- 1) dataset: DIV2K
- 2) data size: 64x64
- 3) batch size: 16
- 4) epoch: 10

## 8. Model 深度

太淺 loss 會降不下去，但是太深也會容易 overfitting，最後是出最好的深度為 16。

| Layer | FLOPs(G) | PARAMS(M) | PSNR   |
|-------|----------|-----------|--------|
| 4     | 5.5045   | 0.024     | 23.748 |
| 16    | 7.5634   | 0.048     | 28.655 |
| 24    | 8.9349   | 0.059     | 28.537 |
| 32    | 10.3064  | 0.069     | 28.2   |

## 9. Upscale 方法

最後選擇 pixel shuffle 在 channel 數為 12 時做，因為已算量較小。在 channel 數為 56 時做 pixel shuffle 雖然會提升 PSNR，但是如果是這樣的話使用 FSRCNN 最原始的方法，不但計算量和參數量較小，PSNR 也較大。

| upscale                 | FLOPs(G) | PARAMS(M) | PSNR   |
|-------------------------|----------|-----------|--------|
| pixel shuffle in 12     | 5.5045   | 0.024     | 23.748 |
| pixel shuffle in 56     | 36.507   | 0.267     | 26.757 |
| transformed convolution | 17.481   | 0.025     | 27.036 |

## 10. Pretrain

在兩個 model 中嘗試了 Pretrain，但是結果都不太理想。但後來也因為使用

pre-trained weights model 可以調整的地方就會減少，所以沒有繼續試。

## 1) Model 3: Residual convolution + HAT

### (1) 結果:

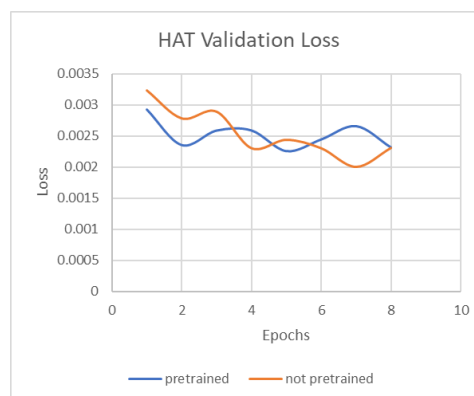
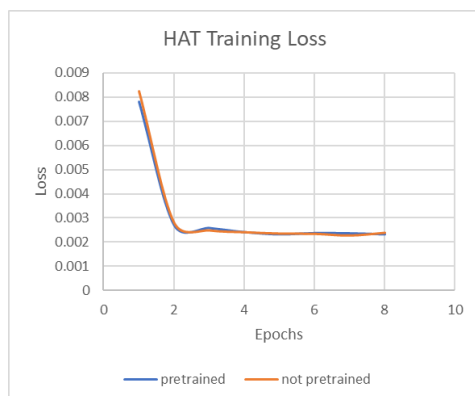
- 由下圖可以看出，**training loss** 幾乎沒有差別，**validation loss** 則是沒有 **pretrain** 時下降的幅度大一些。且有確認過重複的 **layer** 的 **weights** 皆有更新。

### (2) 可能原因:

- **training data** 和 **pretrain** 時用的 **data** 重複。
- 修改過後的 **model** 模型和 **pretrained weights** 不相符，

### (3) 問題點:

- 若要用 **pretrained weight**，**model** 形狀就需要相同，無法改變 **embedding dimension** 等參數，只能改變 **layer** 數。因此即便用了 **HAT-S** 作為 **base model**，計算量仍然非常大，加深會導致 **cuda out of memory**。
- 原本想用只用 **ImageNet pretrain** 過的 **weights**，但是只有找到 **HAT** 有這個 **pretrained weights**，**HAT-S** 沒有。

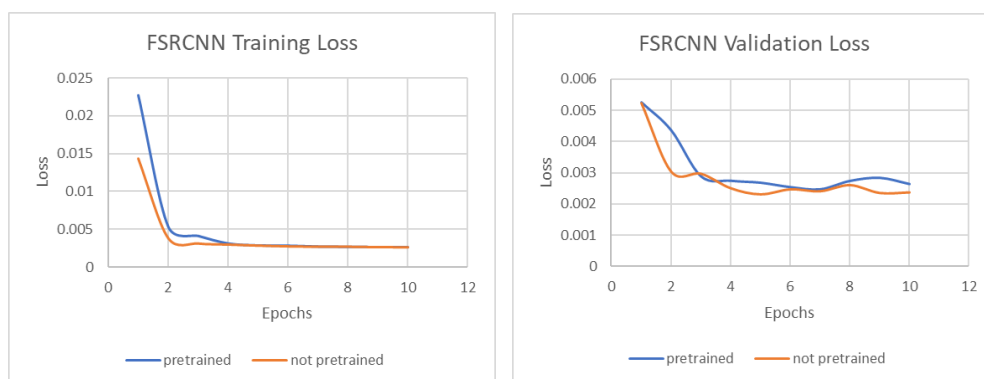


## 2) Model 2: FSRCNN

### (1) 結果:

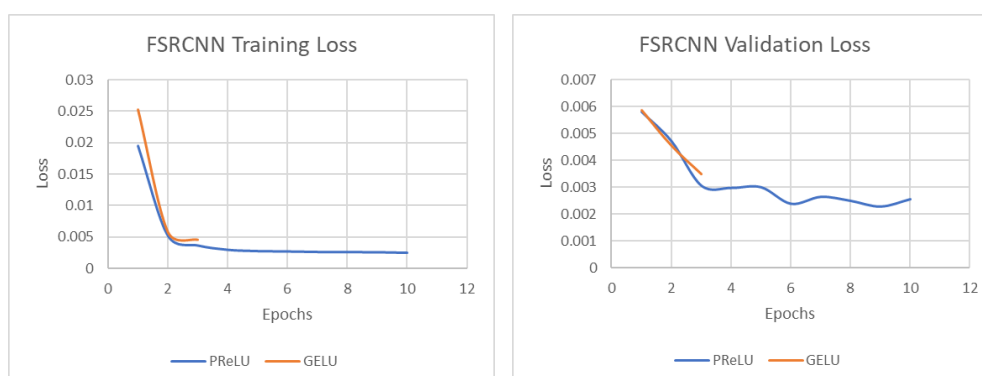
- **PSNR for pretrained / not pretrained: 6.440 / 6.438 dB**
- 可看出兩者 **PSNR** 幾乎沒有差別。且由下圖可知雖然有 **pretrained weights** 時 **training loss** 下降幅度較大，但是最後皆停在差不多的 **loss**。**validation loss** 則是沒有 **pretrain** 時的下降幅度較大。

- (2) 同樣有確認過 **load pretrained weights** 前後的權重有更新，且有確認過 **pretrain** 時使用的 **dataset** 和我訓練時使用的 **dataset** 並沒有重複，因此不太能理解為什麼會沒有用。



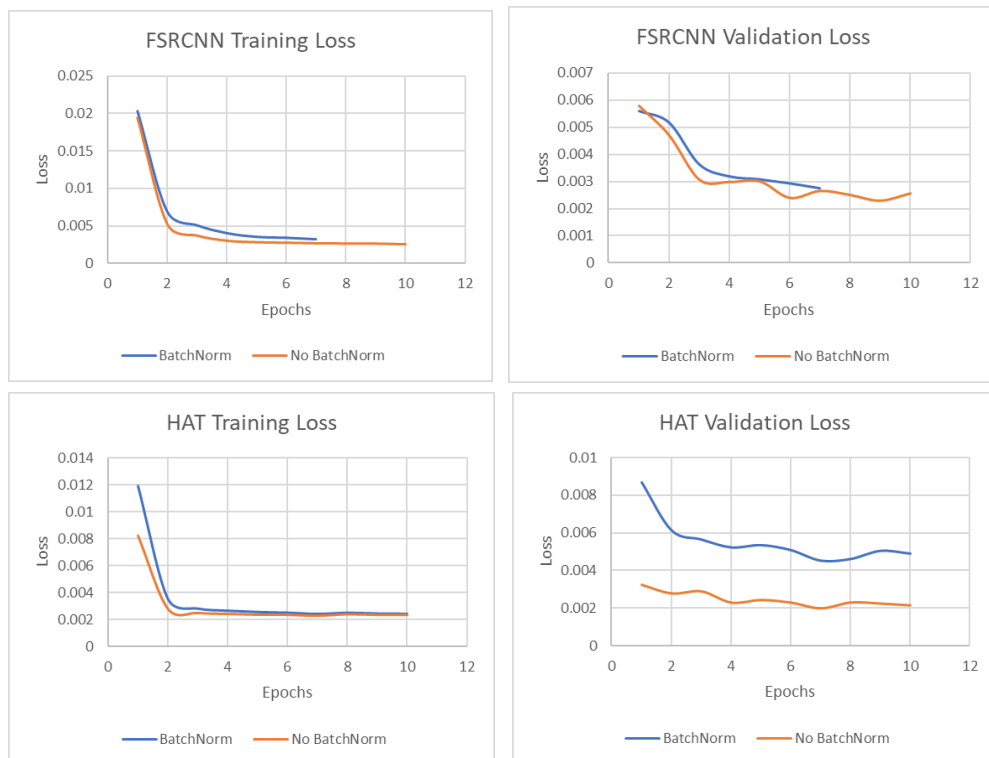
## 11. Activation function

每一個 model 都有自己適合的 activation function，之前訓練時都是 gelu 表現最好，但這次有些不同的結果。以最後使用的 model 使用 gelu 的結果比 PreLU 差一點，後面降不太下去，因此中斷 training。



## 12. Normalization

之前的 task 加上 batch normalization 通常 accuracy 都會提升。而這次由下圖可知，加了 batch normalization 之後，優點是下降較平緩，不會起伏不定，缺點則是下降速度較慢，且計算量會增加。EDSR 的論文中確實也有提到 super resolution 的 model 並不適合加 batch normalization。convolution 加上 hat 的 model 也有相同的情況，而最後印出 PSNR 發現結果幾乎相同，沒有 batch normalization 時 PSNR 為 10.689，有 batch normalization 時則為 10.688。



### 13. Other idea: ClassSR

一種加速 CNN base super resolution 的方法，先用 class module 把圖片按照修復難度分為不同的類別後再進入 super resolution module。由此可以減少 30%到 40%的計算量，且 PSNR 幾乎不變。但是 paper 中提到這並不適合用在圖片大小比較小的 testing data，像是 Set 5，因為分類的過程需要把圖片切小再分類，已經很小的圖片再切小會娶不到特徵，因此不這次的 final project 沒有使用。

### 課程感想

我覺得這一整個暑假的課程對我幫助非常大，讓我從完全沒接觸過機器學習到現在瞭解了什麼是 CNN，什麼是 transformer，還有許多 model 和 training techniques。雖然目前訓練出的 model 結果都不太理想，但和最开始已經有比較有概念了，希望之後可以抓到訓練模型的訣竅。

至於課程建議，我覺得可以再上課的同時開 google meet 給我們錄影，或直接提供上課錄影。因為課程中大部分的知識我們都是第一次接觸，有時候會沒辦法在上課的時候就全部吸收。如過可以有課程錄影的話，回去複習時如過有不懂的地方就可以重複看影片理解。

最後非常謝謝老師和每一位助教的教導，我才能在一個暑假內從零學習到這麼多知識。

### 在 final project 學到什麼

1. 如何自己找 **dataset** 並下載下來 **train**：之前都是助教有提供 **dataset**，但是這次要自己找 **dataset**，上網查要怎麼在 **code** 裡面下載這一些 **dataset**，如何 **unzip**。
2. 依照目標調整 **model**：midterm project 的 **model** 使用基本的 **ResNet** 或 **DenseNet** 等就可以處理，但是 **super resolution** 的模型跟之前的那些模型有許多不同的地方，像是不會一直縮小圖片。另外助教也有提到在 **super resolution** 中模型深度特別重要，不同目標會需要不同特徵的 **model**。
3. 如何調整 **training data**：之前的大部分的時候助教都會在 **code** 裡面引道我們要怎麼處理 **data**，但是這次都要重投自己寫，因此這次我花了非常多時間寫處理 **data** 的 **code**，像是要怎麼調整圖片大小，或是將圖片切小放進 **model train** 之後再組起來等。
4. 讀論文和看 **code**：這次花了非常多時間讀各種英文論文和看比較複雜的 **code**。
5. 使用 **transformer**：我之前的作業都是用 **CNN base** 的 **model**，但是這次中途有嘗試使用 **transformer**。但最後因為使用 **transformer** 時的計算量太大，因此還是用了 **CNN base** 的 **model**。雖然 **transformer** 只要調某些參數，計算量一次就會變非常小，但是我在小型的 **transformer** 下沒有訓練出一個好的 **model**。
6. 使用 **pre-trained weights** 和 **quantization**：因為之前都只有聽過理論，沒有實做過，所以這次是一個很好的經驗讓我動手實作過這兩個常用的技巧。
7. **FLOPs** 不代表運行時間：原本為了節省運行時間而用了 **FLOPs** 較小的 **model**，但後來發現 **FLOPs** 為 12 多的 **model**、36 多的 **model** 和 200 多的 **model** 運行時間一樣，**FLOPs** 不能直接代表訓練所需的時間。
8. 要想清楚之後在訓練：因為 **google colab** 的 **GPU** 用量有限，也很容易斷線，若什麼都拿進去試的話 **GPU** 馬上就會用完了，會友一陣子都沒辦法訓練。還有訓練時間太長的也不要拿進去訓練，因為中間一定會斷線。
9. 加大 **data** 數量才能看出一個 **model** 的好壞：在小的 **dataset** 跑得好，不代表在大的 **dataset** 中也會表現比較好。相對的，在小的 **dataset** 表現不太突出，也不代表在大的 **dataset** 就會表現不好。

Reference:

CoAtNet: <https://arxiv.org/pdf/2106.04803.pdf>

ClassSR: <https://arxiv.org/pdf/2103.04039.pdf>